Assignment Title:

Coursework

Coursework Type:

Individual

Module Name:

ST4005CEM Database System

Intake: September

Submitted By

Shraddha Basnet

Student ID: 240599

CU ID: 15942698

Submitted To

Sunil Thapa

TABLE OF CONTENTS

TABLE OF FIGURES

INTRODUCTION

Database is similar to a digital filing cabinet that is used to manage, store, and arrange data for convenient access. The documentation concepts covered in the database systems course during the second semester are put into practice in this material. This project's main objective is to illustrate important ideas related to normalization, creating SQL queries, and using charts to illustrate data insights. In order to show how database techniques may be successfully applied to real-world data management and processing difficulties across multiple sectors, the project includes processing and illustrating data from a provided CSV file.

NORMALIZATION

Normalization is a technique used to effectively arrange data. Large, disorganized tables can be divided into smaller, more sensible ones with its assistance.

In database, normalization is a procedure used to appropriately arrange data so that:

- No needless duplication or repetition exists.
- The data is accurate, tidy, and manageable.
- The database is less prone to errors and is more efficient.

Phases of normalization

1. UNNORMALIZED FORM (UNF)

Unnormalized Forms (UNFs) can contain complicated or nested data, such as lists inside a single field, and store data in a single table or collection that frequently contains duplication, such as repeated names or addresses.

UNF	LEVEL
CustomerID	1
FullName	1
Address	1
ContactNumber	1
Email	1
CustomerTypeID	1
FutureHost	1
OrderID	2
OrderDate	2
OrderType	2
PartyID	3
PartyDate	3
PartyTime	3
NumberOfGuests	3
ProductID	4
ProductName	4
Description	4
Price	4
OrderDetailsID	5
ProductID	5
Quantity	5
UnitPrice	5
PaymentMethod	5
Amount	5
PaymentDate	5
CategoryID	6
CategoryName	6
Comments	6
SupplierID	7
SupplierName	7
Email	7

Figure 1: UNF

2 FIRST NORMAL FORM (1NF)

In database normalization, the first normal form (1NF) is the fundamental step. Only single, indivisible values—also referred to as atomic values—must be present in each table column in order to meet 1NF. This means that the table should only have one piece of data stored in each cell, and that no columns should have repeating groups or multiple values.

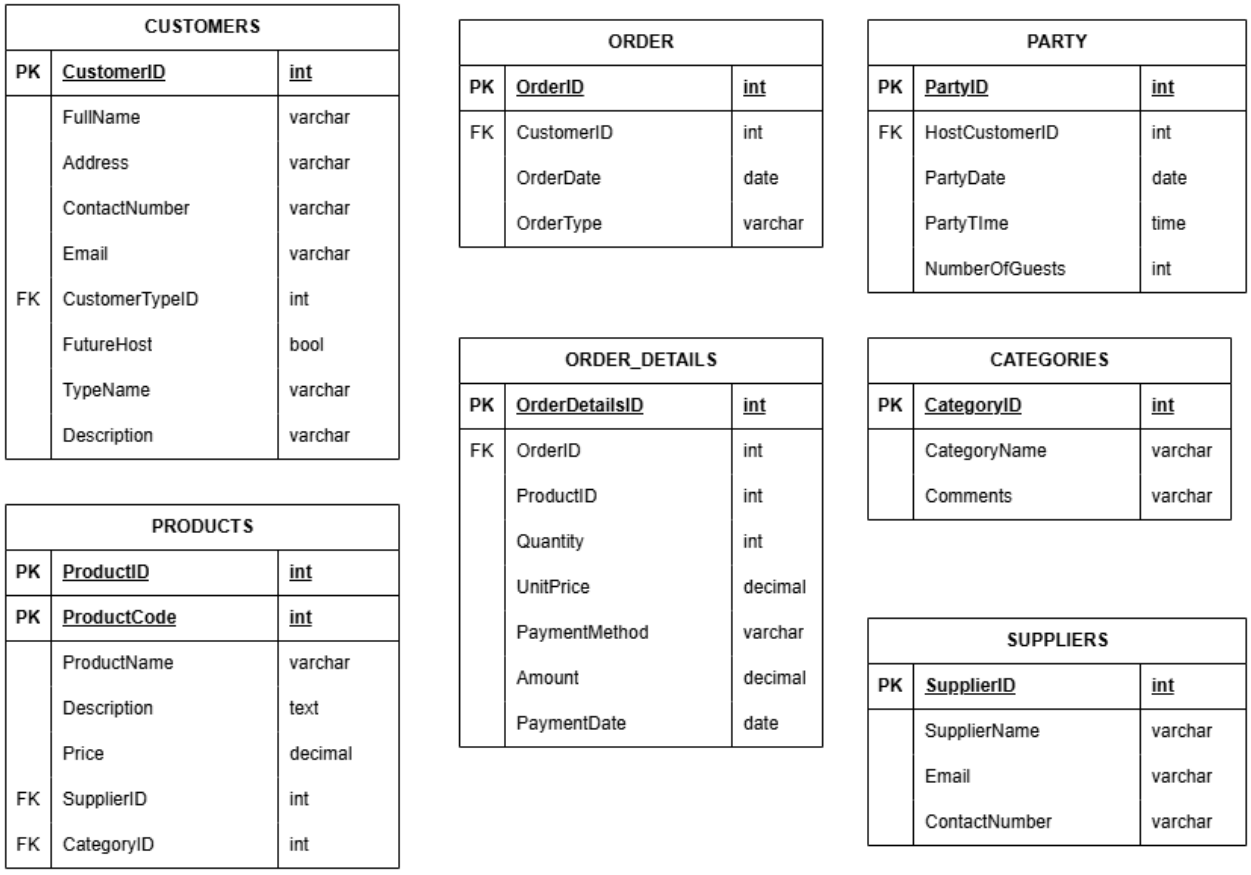


Figure 2: 1NF

3. 2NF

Second Normal Form (2NF) is based on fully functional dependency. A database normalization procedure called Second Normal Form (2NF) further reduces data redundancy and ensures data integrity.

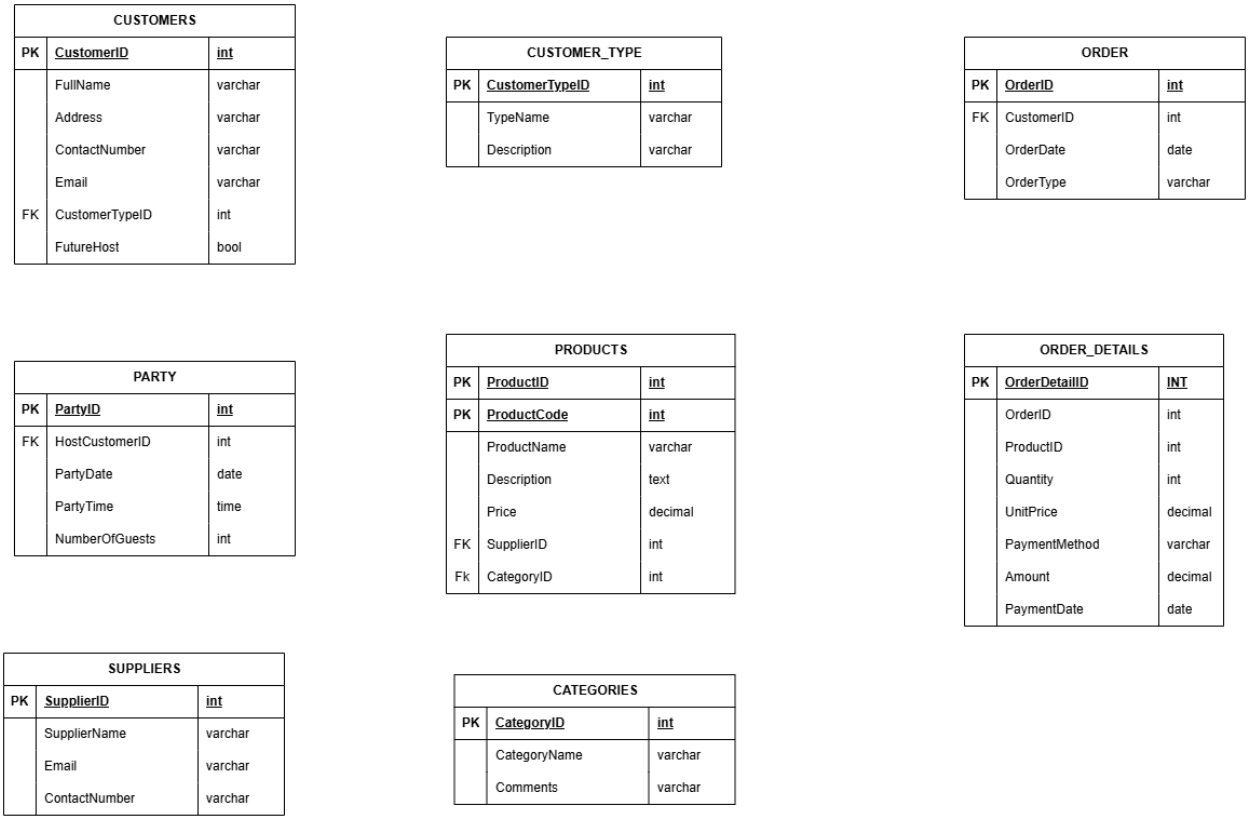


Figure 3: 2NF

4. 3NF

According to 3NF, a table must be in 2NF and all non-key characteristics must rely solely on the primary key and not on any other non-key attributes. This enhances data integrity and eliminates transitive dependencies.

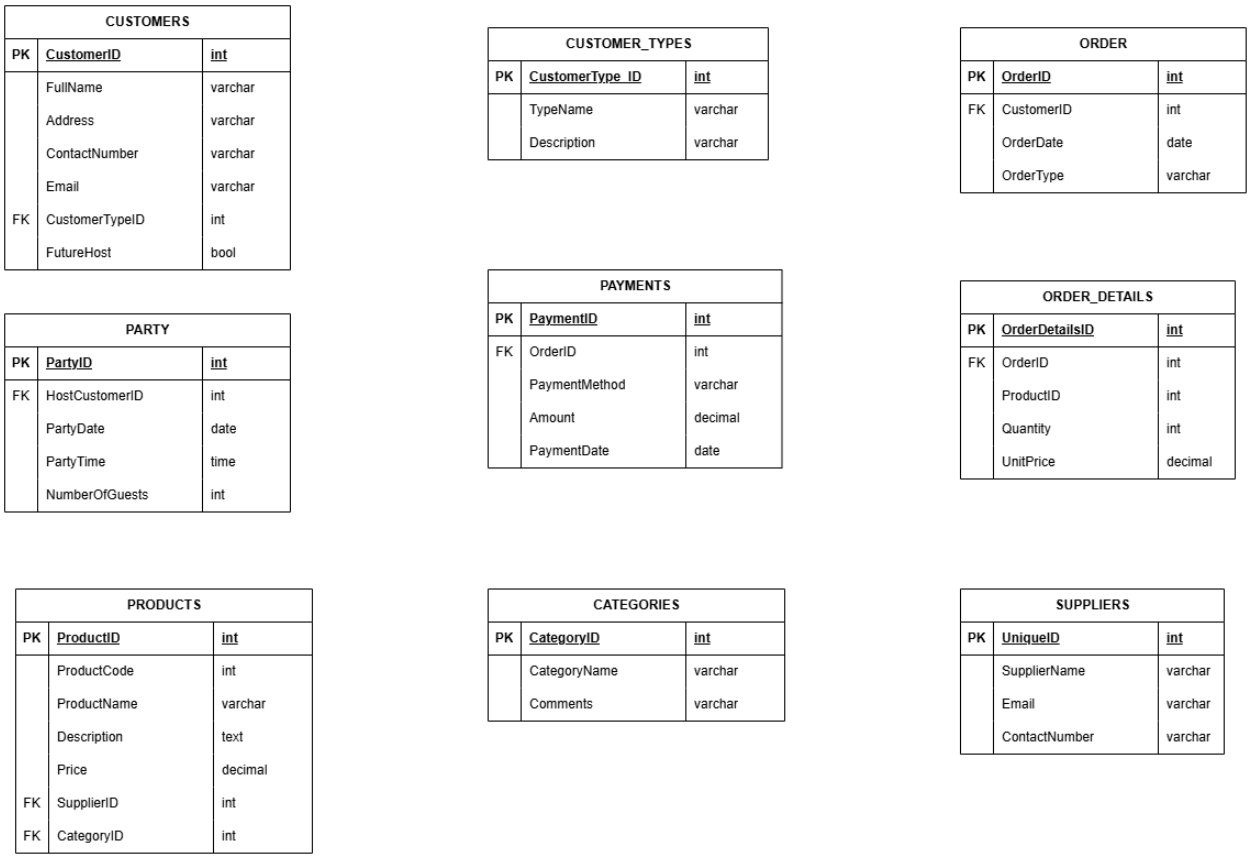


Figure 4: 3NF

ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM)

An entity-relationship (ER) diagram is a diagram of a database's structure. It displays the system's primary entities, or items or objects, together with their attributes, or specifics about them, and the connections between them.

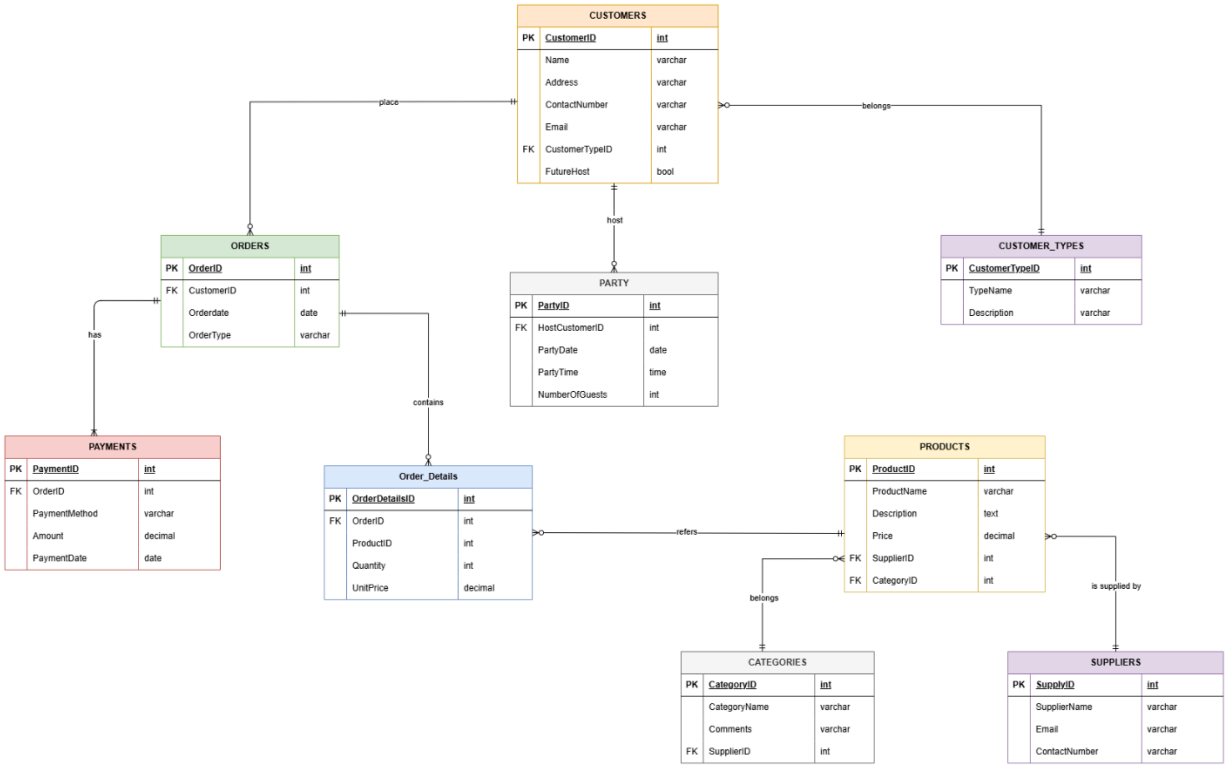


Figure 5: ER Diagram

SQL QUERIES

SQL queries are structured query language (SQL) instructions used to interact with and control data in databases.

1.The Generated Tables

```
CREATE TABLE CUSTOMER_TYPES (
    CustomerTypeID INT PRIMARY KEY AUTO_INCREMENT,
    TypeName VARCHAR(50) NOT NULL,
    Description VARCHAR(255)
);
```

Figure 6: create table / customer types

```
CREATE TABLE CUSTOMERS (
    CustomerID INT PRIMARY KEY AUTO_INCREMENT,
    FullName VARCHAR(100) NOT NULL,
    Address VARCHAR(255),
    ContactNumber VARCHAR(20),
    Email VARCHAR(100),
    CustomerTypeID INT,
    FutureHost BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (CustomerTypeID) REFERENCES CUSTOMER_TYPES(CustomerTypeID)
);
```

Figure 7: customers

```sql
CREATE TABLE PARTY (
    PartyID INT PRIMARY KEY AUTO_INCREMENT,
    HostCustomerID INT,
    PartyDate DATE,
    PartyTime TIME,
    NumberOfGuests INT,
    FOREIGN KEY (HostCustomerID) REFERENCES CUSTOMERS(CustomerID)
);
```

Figure 8: party

```sql
CREATE TABLE SUPPLIERS (
    SupplierID INT PRIMARY KEY AUTO_INCREMENT,
    SupplierName VARCHAR(100),
    Email VARCHAR(100),
    ContactNumber VARCHAR(20)
);
```

Figure 9: suppliers

```sql
CREATE TABLE CATEGORIES (
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    CategoryName VARCHAR(50),
    Comments VARCHAR(255)
);
```

Figure 10: categories

```sql
CREATE TABLE PRODUCTS (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductCode INT,
    ProductName VARCHAR(100),
    Description TEXT,
    Price DECIMAL(10,2),
    SupplierID INT,
    CategoryID INT,
    FOREIGN KEY (SupplierID) REFERENCES SUPPLIERS(SupplierID),
    FOREIGN KEY (CategoryID) REFERENCES CATEGORIES(CategoryID)
);
```

Figure 11: products

```sql
CREATE TABLE ORDERS (
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    CustomerID INT,
    OrderDate DATE,
    OrderType VARCHAR(50),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMERS(CustomerID)
);
```

Figure 12: orders

```
CREATE TABLE ORDER_DETAILS (
    OrderDetailsID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    ProductID INT,
    Quantity INT,
    UnitPrice DECIMAL(10,2),
    FOREIGN KEY (OrderID) REFERENCES ORDERS (OrderID),
    FOREIGN KEY (ProductID) REFERENCES PRODUCTS(ProductID)
);
```

Figure 13: order details

```
CREATE TABLE PAYMENTS (
    PaymentID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    PaymentMethod VARCHAR(50),
    Amount DECIMAL(10,2),
    PaymentDate DATE,
    FOREIGN KEY (OrderID) REFERENCES ORDERS (OrderID)
);
```

Figure 14: payments

2 Data that are Inserted

```
CREATE TABLE CUSTOMER_TYPES (
    CustomerTypeID INT PRIMARY KEY AUTO_INCREMENT,
    TypeName VARCHAR(50) NOT NULL,
    Description VARCHAR(255)
);
```

```
-- CUSTOMER_TYPES
INSERT INTO CUSTOMER_TYPES (TypeName, Description) VALUES
('Customer', 'Standard price'),
('Host', '-2%'),
('Ambassador', '15% of profit');
13
14 •   select * from customer_types;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerTypeID	TypeName	Description
1	Customer	Standard price
2	Host	-2%
3	Ambassador	15% of profit
NULL	NULL	NULL

Figure 15: enter data / customer types

```
CREATE TABLE CUSTOMERS (
    CustomerID INT PRIMARY KEY AUTO_INCREMENT,
    FullName VARCHAR(100) NOT NULL,
    Address VARCHAR(255),
    ContactNumber VARCHAR(20),
    Email VARCHAR(100),
    CustomerTypeID INT,
    FutureHost BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (CustomerTypeID) REFERENCES CUSTOMER_TYPES(CustomerTypeID)
);
```

```
-- CUSTOMERS
INSERT INTO CUSTOMERS (FullName, Address, PhoneNumber, Email, CustomerTypeID, FutureHost) VALUES
('Krish Rai', 'Jhapa', '9845641792', 'krish@hmail.com', (SELECT CustomerTypeID FROM CUSTOMER_TYPES WHERE TypeName='Host'), FALSE),
('Anish Kc', 'Chitwan', '9844532468', 'anish@gmail.com', (SELECT CustomerTypeID FROM CUSTOMER_TYPES WHERE TypeName='Customer'), TRUE),
('Om Thapa', 'Pokhara', '9845431792', 'om12@cmail.com', (SELECT CustomerTypeID FROM CUSTOMER_TYPES WHERE TypeName='Ambassador'), FALSE),
('Raj Joshi', 'Butwal', '9845124793', 'raj@smail.com', (SELECT CustomerTypeID FROM CUSTOMER_TYPES WHERE TypeName='Customer'), FALSE);
```

```
36
37 •   select * from customers;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	FullName	Address	ContactNumber	Email	CustomerTypeID	FutureHost
1	Krish Rai	Jhapa	9845641792	krish@hmail.com	2	0
2	Anish Kc	Chitwan	9844532468	anish@gmail.com	1	1
3	Om Thapa	Pokhara	9845431792	om12@cmail.com	3	0
4	Raj Joshi	Butwal	9845124793	raj@smail.com	1	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 16: customers

```
CREATE TABLE PARTY (
    PartyID INT PRIMARY KEY AUTO_INCREMENT,
    HostCustomerID INT,
    PartyDate DATE,
    PartyTime TIME,
    NumberOfGuests INT,
    FOREIGN KEY (HostCustomerID) REFERENCES CUSTOMERS(CustomerID)
);

-- PARTY
INSERT INTO PARTY (PartyID, HostCustomerID, PartyDate, PartyTime, NumberOfGuests) VALUES
(17, (SELECT CustomerID FROM CUSTOMERS WHERE FullName = 'Krish Rai'), '2018-01-12', '20:00:00', 23),
(14, (SELECT CustomerID FROM CUSTOMERS WHERE FullName = 'Anish Kc'), '2018-03-24', '19:30:00', 11),
(34, (SELECT CustomerID FROM CUSTOMERS WHERE FullName = 'Om Thapa'), '2018-04-08', '11:00:00', 8),
(67, (SELECT CustomerID FROM CUSTOMERS WHERE FullName = 'Raj Joshi'), '2018-04-25', '15:00:00', 12);
```

```
57
58 •   select * from party;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

PartyID	HostCustomerID	PartyDate	PartyTime	NumberOfGuests
14	2	2018-03-24	19:30:00	11
17	1	2018-01-12	20:00:00	23
34	3	2018-04-08	11:00:00	8
67	4	2018-04-25	15:00:00	12
NULL	NULL	NULL	NULL	NULL

Figure 17: party

```sql
CREATE TABLE SUPPLIERS (
    SupplierID INT PRIMARY KEY AUTO_INCREMENT,
    SupplierName VARCHAR(100),
    Email VARCHAR(100),
    ContactNumber VARCHAR(20)
);

-- SUPPLIERS
INSERT INTO SUPPLIERS (SupplierID, SupplierName, Email, ContactNumber) VALUES
(12, 'ABC', 'supplier@abc.net', '1282456762'),
(1, 'CDE', 'supplier@cde.co', '1252776789'),
(34, 'DEF', 'supplier@def.com', '1705674890'),
(2, 'GHI', 'supplier@ghi.com.np', '1200467876');
74
75 •   select * from suppliers;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

SupplierID	SupplierName	Email	ContactNumber
1	CDE	supplier@cde.co	1252776789
2	GHI	supplier@ghi.com.np	1200467876
12	ABC	supplier@abc.net	1282456762
34	DEF	supplier@def.com	1705674890
NULL	NULL	NULL	NULL

Figure 18: suppliers

```sql
CREATE TABLE CATEGORIES (
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    CategoryName VARCHAR(50),
    Comments VARCHAR(255)
);
 -- CATEGORIES
 INSERT INTO CATEGORIES (CategoryID, CategoryName, Comments) VALUES
 (1, 'Skincare', 'Summer Season'),
 (2, 'Bodycare', 'Summer Range'),
 (3, 'Makeup', 'All minerals'),
 (4, 'Haircare', 'All products');
91
92 •   select * from categories;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CategoryID	CategoryName	Comments
1	Skincare	Summer Season
2	Bodycare	Summer Range
3	Makeup	All minerals
4	Haircare	All products
NULL	NULL	NULL

Figure 19: categories

```sql
CREATE TABLE PRODUCTS (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductCode INT,
    ProductName VARCHAR(100),
    Description TEXT,
    Price DECIMAL(10,2),
    SupplierID INT,
    CategoryID INT,
    FOREIGN KEY (SupplierID) REFERENCES SUPPLIERS(SupplierID),
    FOREIGN KEY (CategoryID) REFERENCES CATEGORIES(CategoryID)
);

-- PRODUCTS

INSERT INTO PRODUCTS (ProductCode, ProductName, Description, Price, SupplierID, CategoryID) VALUES
-- ProductCode column is missing in your schema; I'll assume ProductName replaces it
(123, 'ABC + free eye refresh', 'Cleanser- Natural Ingredients', 52.00, 12, 1),
(23, 'Travel Essentials', 'Shampoo and Conditioner', 30.00, 1, 4),
(234, 'Smooth Skin Collection', 'Skincare Products – Cleanser, Toner, Moisturiser', 45.00, 34, 1),
(235, 'Super Deluxe Collection', 'Shower Collection', 35.00, 2, 2);
```

```sql
119
120 •   select * from products;
121
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ProductID	ProductCode	ProductName	Description	Price	SupplierID	CategoryID
1	123	ABC + free eye refresh	Cleanser- Natural Ingredients	52.00	12	1
2	23	Travel Essentials	Shampoo and Conditioner	30.00	1	4
3	234	Smooth Skin Collection	Skincare Products – Cleanser, Toner, Moisturiser	45.00	34	1
4	235	Super Deluxe Collection	Shower Collection	35.00	2	2
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 20: products

```sql
CREATE TABLE ORDERS (
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    CustomerID INT,
    OrderDate DATE,
    OrderType VARCHAR(50),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMERS(CustomerID)
);

-- ORDERS
INSERT INTO ORDERS (OrderID, CustomerID, OrderDate, OrderType) VALUES
(1, 15, '2018-01-12', 'Email'),
(2, 16, '2018-03-24', 'Party'),
(3, 14, '2018-04-08', 'Party'),
(4, 15, '2018-04-25', 'Email');
```

```sql
139
140 •   select * from orders;
141
142
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

OrderID	CustomerID	OrderDate	OrderType
1	2	2018-01-12	Email
2	3	2018-03-24	Party
3	1	2018-04-08	Party
4	2	2018-04-25	Email
NULL	NULL	NULL	NULL

Figure 21: orders

```sql
CREATE TABLE ORDER_DETAILS (
    OrderDetailsID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    ProductID INT,
    Quantity INT,
    UnitPrice DECIMAL(10,2),
    FOREIGN KEY (OrderID) REFERENCES ORDERS (OrderID),
    FOREIGN KEY (ProductID) REFERENCES PRODUCTS(ProductID)
);


-- ORDER_DETAILS
INSERT INTO ORDER_DETAILS (OrderID, ProductID, Quantity, UnitPrice) VALUES
(1, (SELECT ProductID FROM PRODUCTS WHERE ProductName = 'ABC + free eye refresh'), 25, 52.00),
(2, (SELECT ProductID FROM PRODUCTS WHERE ProductName = 'Travel Essentials'), 1, 30.00),
(3, (SELECT ProductID FROM PRODUCTS WHERE ProductName = 'Super Deluxe Collection'), 1, 35.00),
(4, (SELECT ProductID FROM PRODUCTS WHERE ProductName = 'Travel Essentials'), 5, 30.00);
```

```sql
170
171 •   select * from order_details;
172
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

OrderDetailsID	OrderID	ProductID	Quantity	UnitPrice
1	1	1	25	52.00
2	2	2	1	30.00
3	3	4	1	35.00
4	4	2	5	30.00
NULL	NULL	NULL	NULL	NULL

Figure 22: order details

```sql
CREATE TABLE PAYMENTS (
    PaymentID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    PaymentMethod VARCHAR(50),
    Amount DECIMAL(10,2),
    PaymentDate DATE,
    FOREIGN KEY (OrderID) REFERENCES ORDERS (OrderID)
);
-- PAYMENTS
INSERT INTO PAYMENTS (OrderID, PaymentMethod, Amount, PaymentDate) VALUES
(1, 'Credit Card', 1300.00, '2018-01-12'),
(2, 'Cash', 30.00, '2018-03-24'),
(3, 'Cheque', 35.00, '2018-04-08'),
(4, 'Cash', 150.00, '2018-04-25');
```

```
200
201 •   select * from payments;
```

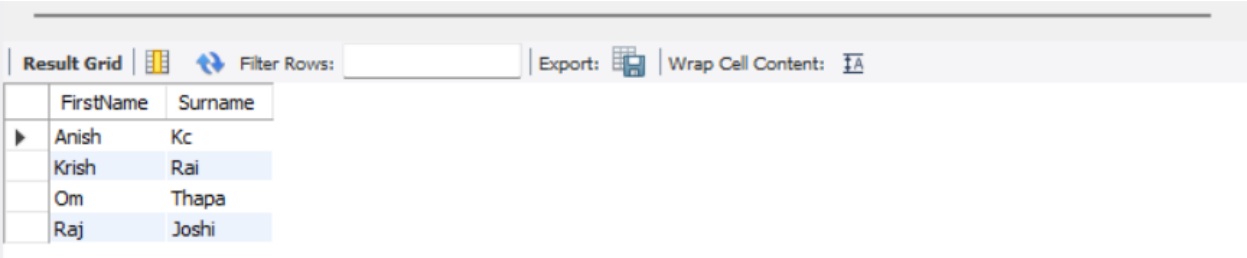
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

PaymentID	OrderID	PaymentMethod	Amount	PaymentDate
1	1	Credit Card	1300.00	2018-01-12
2	2	Cash	30.00	2018-03-24
3	3	Cheque	35.00	2018-04-08
4	4	Cash	150.00	2018-04-25
NULL	NULL	NULL	NULL	NULL

Figure 23: payments

3. Write a query that selects the first name and surname of customers alphabetical order of first name

```
SELECT
    SUBSTRING_INDEX(FullName, ' ', 1) AS FirstName,
    SUBSTRING_INDEX(FullName, ' ', -1) AS Surname
FROM CUSTOMERS
ORDER BY FirstName ASC;
```



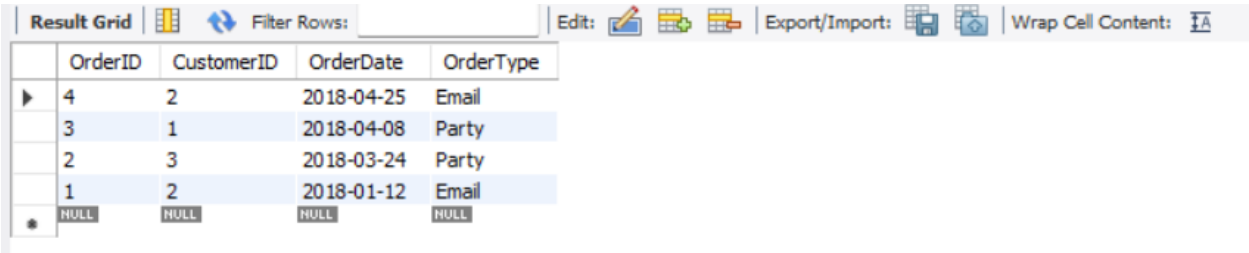
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FirstName	Surname
Anish	Kc
Krish	Rai
Om	Thapa
Raj	Joshi

Figure 24: Alphabetical-order

4.Write a query that selects all orders by date in descending order

```
SELECT *
FROM ORDERS
ORDER BY OrderDate DESC;
```



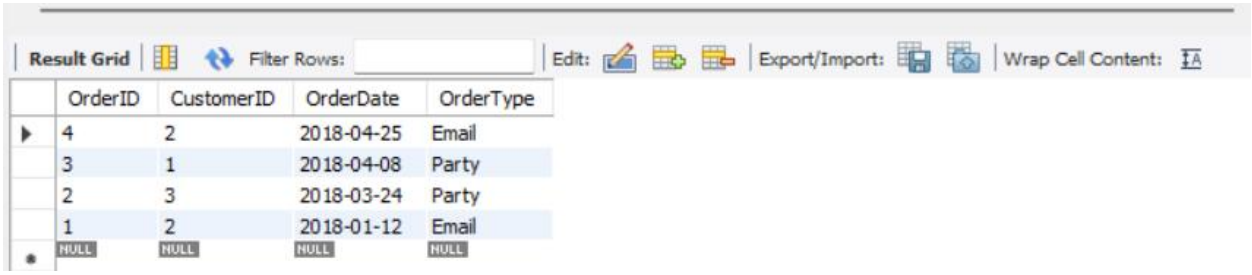
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

OrderID	CustomerID	OrderDate	OrderType
4	2	2018-04-25	Email
3	1	2018-04-08	Party
2	3	2018-03-24	Party
1	2	2018-01-12	Email
NULL	NULL	NULL	NULL

Figure 25: Descending-order

5. Write a query that counts all the orders where the payment method is cash

```
SELECT COUNT(*) AS CashOrderCount
FROM PAYMENTS
WHERE PaymentMethod = 'Cash';
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

OrderID	CustomerID	OrderDate	OrderType
4	2	2018-04-25	Email
3	1	2018-04-08	Party
2	3	2018-03-24	Party
1	2	2018-01-12	Email
NULL	NULL	NULL	NULL

Figure 26: count

6. Write a query that returns all party orders and date ordered where quantity is greater than 25

```
226 •   SELECT o.OrderID, o.OrderDate
227     FROM ORDERS o
228     JOIN ORDER_DETAILS od ON o.OrderID = od.OrderID
229     WHERE o.OrderType = 'Party' AND od.Quantity > 25;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	OrderDate

Figure 27: Quantity > 25

7. Update email addresses of suppliers. For example: ABC to abc-supplier@abc.net

```
232 •  UPDATE SUPPLIERS
233    SET Email = 'abc-supplier@abc.net'
234    WHERE SupplierID = 12;
235 •  select * from suppliers;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

SupplierID	SupplierName	Email	ContactNumber
1	CDE	supplier@cde.co	1252776789
2	GHI	supplier@ghi.com.np	1200467876
12	ABC	abc-supplier@abc.net	1282456762
34	DEF	supplier@def.com	1705674890
NULL	NULL	NULL	NULL

Figure 28: update-email

8. Add a new category 'Healthy Eating' to the category table

```
238 •  INSERT INTO CATEGORIES (CategoryNme, Comments)
239    VALUES ('Healthy Eating', 'Category for healthy food products');
240 •  select * from categories;
241
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CategoryID	CategoryName	Comments
1	Skincare	Summer Season
2	Bodycare	Summer Range
3	Makeup	All minerals
4	Haircare	All products
NULL	NULL	NULL

Figure 29: New-category

9. Update the customer type for Krishna Rai from Host to Ambassador

```
242 •   UPDATE CUSTOMERS
243     SET CustomerTypeID = 3
244     WHERE CustomerTypeID = 2;
245
246 •   select * from customers;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	FullName	Address	ContactNumber	Email	CustomerTypeID	FutureHost
1	Krish Rai	Jhapa	9845641792	krish@hmail.com	3	0
2	Anish Kc	Chitwan	9844532468	anish@gmail.com	1	1
3	Om Thapa	Pokhara	9845431792	om12@cmail.com	3	0
4	Raj Joshi	Butwal	9845124793	raj@smail.com	1	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 30: update-customertypeid

10. Delete the product 'Smooth Skin Collection

```
255 •   SELECT ProductID FROM PRODUCTS WHERE ProductName = 'Smooth Skin Collection';
256 •   DELETE FROM PRODUCTS WHERE ProductID = 3;
257 •   SELECT * FROM PRODUCTS;
258
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ProductID	ProductCode	ProductName	Description	Price	SupplierID	CategoryID
1	123	ABC + free eye refresh	Cleanser- Natural Ingredients	52.00	12	1
2	23	Travel Essentials	Shampoo and Conditioner	30.00	1	4
4	235	Super Deluxe Collection	Shower Collection	35.00	2	2
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 31: Delete-product

DATA VISUALIZATION

In conclusion, this course gave students a strong foundation in database administration and design. I developed a thorough understanding of how to efficiently organize, handle, and comprehend data through tasks including building SQL tables, ER diagrams, performing normalization, and visualizing data. Data visualization is widely used in business, research, and many other industries since it saves time and increases the accuracy of data interpretation.

The objective of data visualization is to make complex data easier to comprehend and analyze by presenting it in an understandable and visually appealing manner.

```
from google.colab import files
uploaded = files.upload()
```

```
import pandas as pd
df = pd.read_csv("googleplaystore(in).csv")
df_cleaned = df.copy()
df.head()
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	7-Jan-18	1.0.0	4.0.3 and up
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone	Art & Design;Pretend Play	15-Jan-18	2.0.0	4.0.3 and up
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7M	5,000,000+	Free	0	Everyone	Art & Design	1-Aug-18	1.2.4	4.0.3 and up
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	215644	25M	50,000,000+	Free	0	Teen	Art & Design	8-Jun-18	Varies with device	4.2 and up
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	967	2.8M	100,000+	Free	0	Everyone	Art & Design;Creativity	20-Jun-18	1.1	4.4 and up

```
df_cleaned = df.drop_duplicates()
df['Reviews'] = pd.to_numeric(df['Reviews'],errors='coerce')
print(df_cleaned)
```

```
if df_cleaned['Rating'].dtype in ['float64', 'int64']:
    df_cleaned['Rating']=df_cleaned['Rating'].fillna(df_cleaned['Rating'].mean())
    print(df_cleaned)
```

```
df_cleaned['Reviews']= pd.to_numeric(df_cleaned['Reviews'],errors='coerce').fillna(0).astype(int)
df_cleaned['Reviews']= df_cleaned['Reviews'].apply(lambda x: f"{x:,}")
print(df_cleaned)
```

```
[ ]  print(df_cleaned)
```

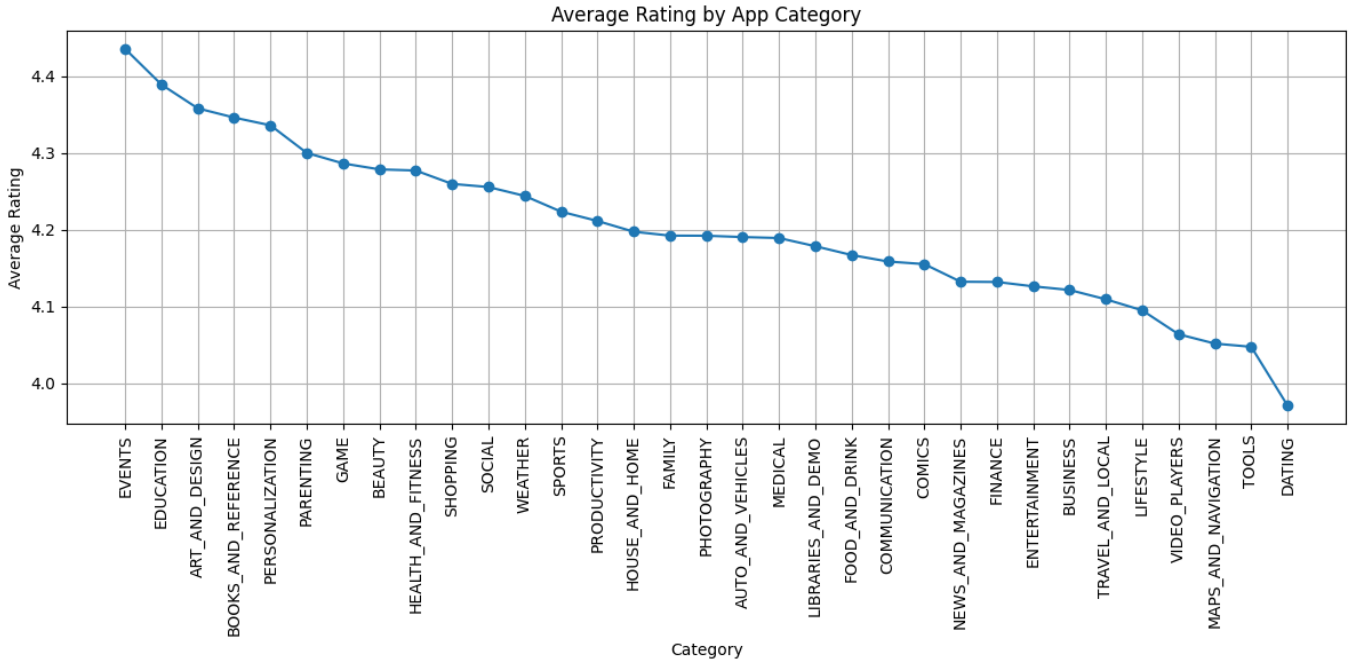
```
import numpy as np

def size_to_bytes(size):
  if isinstance(size, str):
    size=size.strip().lower()
    if size.endswith('m'):
      return int(float(size.replace('m', ''))*1_00_000)
    elif size.endswith('k'):
      return int(float(size.replace('k', ''))*1_000)
    return np.nan

#Convert "Varies with device " to NaN
df_cleaned['Size']=df_cleaned['Size'].replace('Varies with device', np.nan)
#Apply conversion to bytes
df_cleaned['Size']=df_cleaned['Size'].apply(size_to_bytes)

#Format with commas
df_cleaned['Size']=df_cleaned['Size'].apply(lambda x: f"{x:,}"if pd.notnull(x)else "Unknown")
```

```
df_cleaned['Price']=df_cleaned['Price'].str.replace('$', '',regex=False)
df_cleaned['Price']=pd.to_numeric(df_cleaned['Price'], errors='coerce')
print(df_cleaned)
```

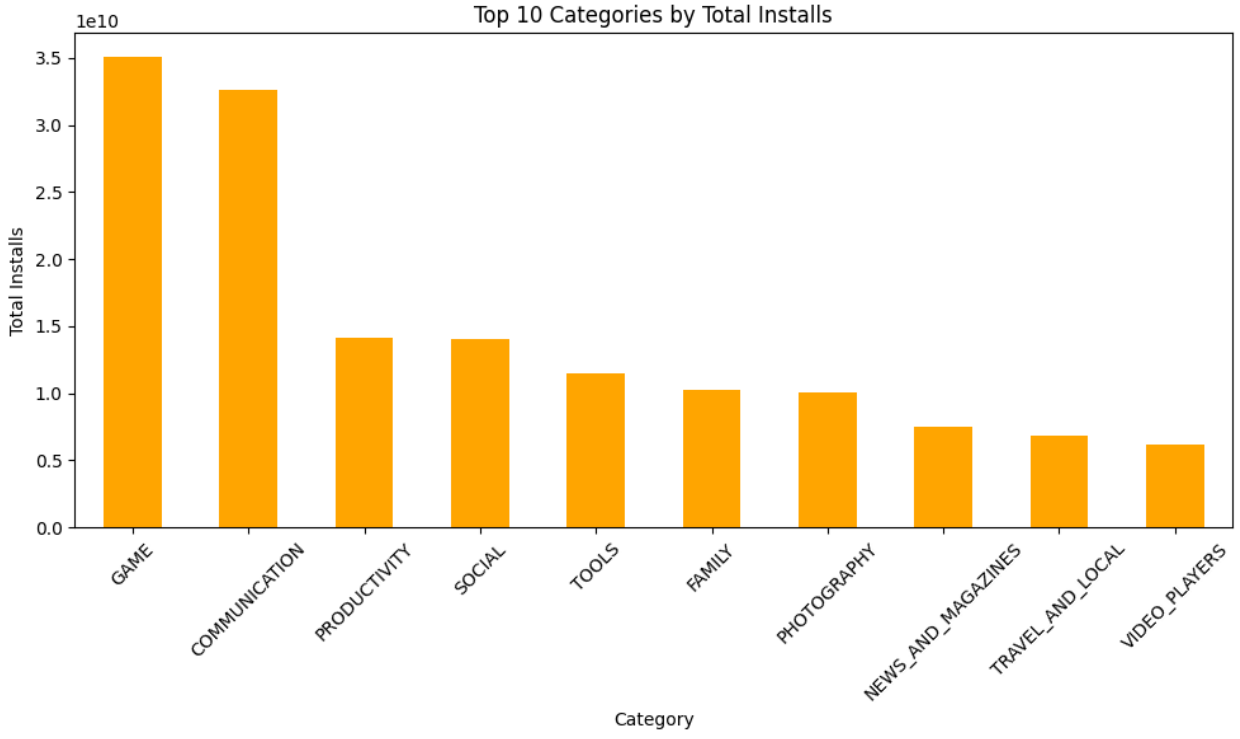
```
df_cleaned['Last Updated']=pd.to_datetime(df_cleaned['Last Updated'],errors='coerce')
print(df_cleaned)
```

```
import pandas as pd
import numpy as np

df_cleaned = pd.read_csv("googleplaystore(in).csv")

df_cleaned['Installs']=df_cleaned['Installs'].astype(str)
df_cleaned['Installs']=df_cleaned['Installs'].apply(
    lambda x: x if isinstance(x, str) and x.replace(',','').replace('+','').isdigit() else np.nan
)
df_cleaned['Installs']=df_cleaned['Installs'].str.replace('[+,]','',regex=True)

df_cleaned['Installs']=pd.to_numeric(df_cleaned['Installs'],errors='coerce')
df_cleaned=df_cleaned[df_cleaned['Installs'].notna()]
print(df_cleaned)
```

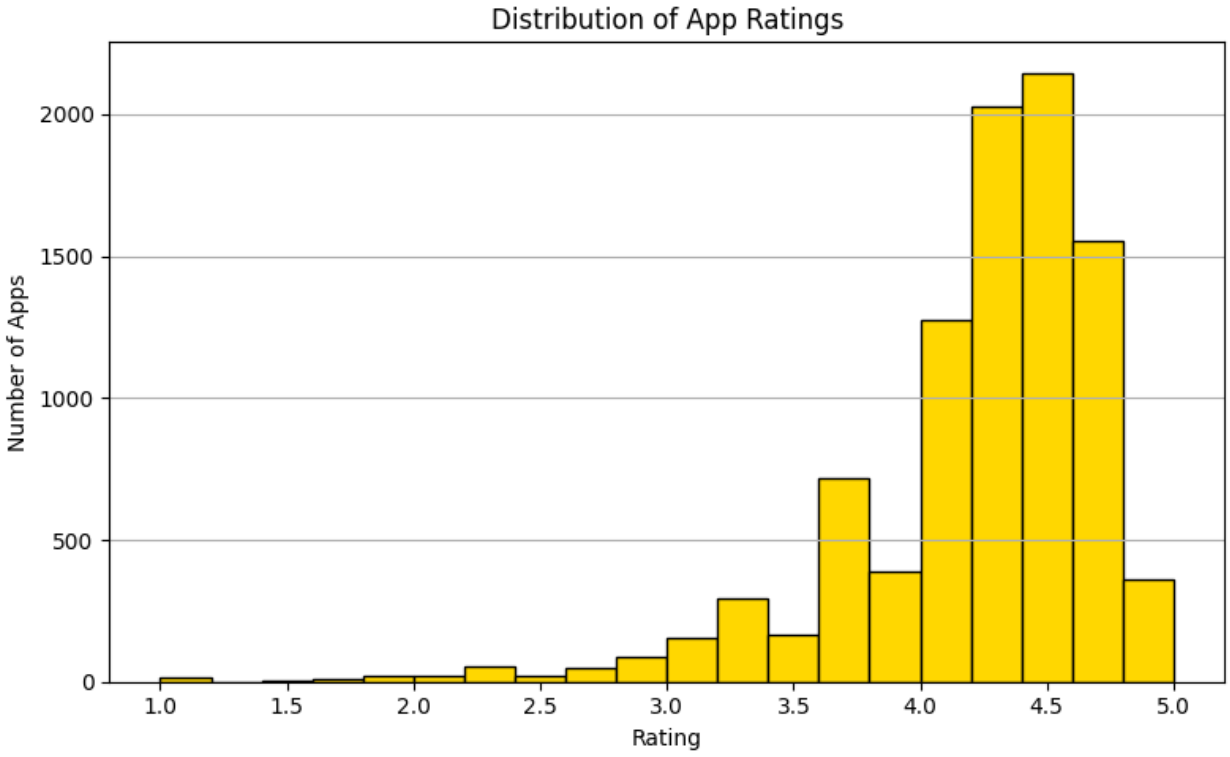
```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("googleplaystore(in).csv")

# Remove ratings that are not between 1 and 5
df = df[(df['Rating'] >= 1.0) & (df['Rating'] <= 5.0)]

# Line graph of average rating per category
category_rating = df.groupby('Category')['Rating'].mean().sort_values(ascending = False)

plt.figure(figsize=(12,6))
plt.plot(category_rating.index, category_rating.values, marker = 'o', linestyle = '-')
plt.title('Average Rating by App Category')
plt.xlabel('Category')
plt.ylabel('Average Rating')
plt.xticks(rotation = 90)
plt.grid(True)
plt.tight_layout()
plt.show()
```



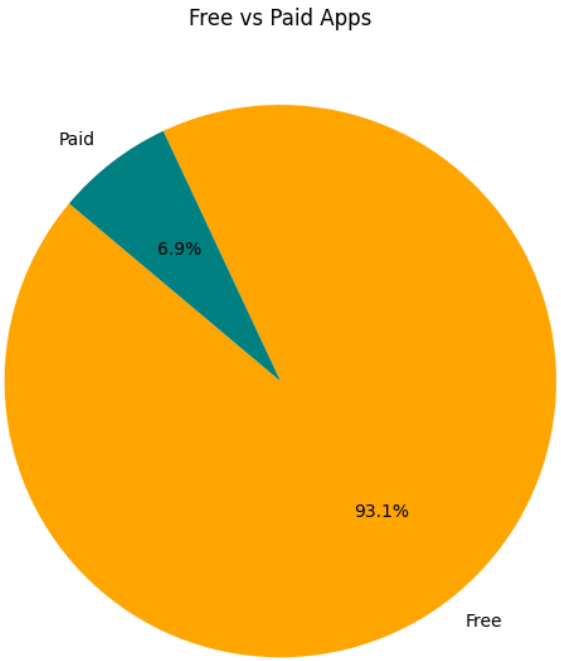
```python
# Clean the 'Installs' column
df['Installs'] = df['Installs'].str.replace(',', '').str.replace('+', '', regex=False)
df['Installs'] = pd.to_numeric(df['Installs'], errors='coerce')  # convert to numeric, set invalid to NaN

# Now group and plot
top_installs = df.groupby('Category')['Installs'].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(10, 6))
top_installs.plot(kind='bar', color='orange')
plt.title('Top 10 Categories by Total Installs')
plt.xlabel('Category')
plt.ylabel('Total Installs')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```python
# Histogram of ratings
plt.figure(figsize=(8,5))
df['Rating'].dropna().plot(kind ='hist', bins= 20, color= 'gold', edgecolor='black')
plt.title('Distribution of App Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Apps')
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```




```python
# Pie chart of free vs paid
type_counts = df['Type'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(type_counts,labels=type_counts.index, autopct='%1.1f%%', startangle=140, colors =['orange','teal'])
plt.title('Free vs Paid Apps')
plt.tight_layout()
plt.show()
```


Free vs Paid Apps
Paid
6.9%
93.1%
Free

CONCLUSION

To sum up, this course gave students a thorough understanding of database system design and administration. I developed a strong grasp of how to efficiently organize, store, and analyze data by building SQL tables, generating an Entity-Relationship (ER) diagram, using normalization techniques, and visualizing data. These abilities have improved my capacity to create organized databases and effectively communicate data insights, both of which are critical for efficient data administration and decision-making.

REFERENCES

DATABASE NORMALIZATION DESIGN PATTERN. (n.d.). IEEE Xplore. https://ieeexplore.ieee.org/abstract/document/8251067

Desyilal. (n.d.). FUNDAMENTALS OF DATABASE SYSTEM -MODULE -CHAPTER_ONE. Scribd. https://www.scribd.com/document/574431366/Fundamentals-of-Database-System-Module-CHapter-One

(n.d.). Forbidden. https://d1wqtxts1xzle7.cloudfront.net/37950350/5313ijdms03-libre.pdf?1434779031=&response-content-disposition=inline%3B+filename%3DALGORITHM_FOR_RELATIONAL_DATABASE_NORMAL.pdf&Expires=1752594810&Signature=CW0MATl7IU7xnIKDu2V-Zs0eqkiqmxV~-Tzd1flfBjHtfOgKTZdwQXdFgEPJIczHomjaCWMqilOJmSm0NItr03ioALXjcXWadvh~nNBaE~lv2Ctah5JpOBa~IXKsHYasUyUu0mEHtmYQtwB5zrRcqyHk~b4Qg0Y6YtntYpCFKSj7TEUGWn1~eNP3g1B1fWjhBE1mwl8xHArLR3~4pP1ufUAUtl6mXD5RBe8P2edPSD8R8GI~zWzmZjok9qjB2YoYa3WwXLq0KOFLI1c4WwTnQsaa7vDGFy6iZNgj6Vhxg3YNIGEZadwyGBbjK2SkTWaeQ14XWGYkiDZefRxqELBcpQ__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA

HOW TO DRAW AN ER DIAGRAM: A STEP-BY-STEP GUIDE | MIRO. (n.d.). https://miro.com/. https://miro.com/diagramming/how-to-draw-an-er-diagram/

INTRODUCTION OF ER MODEL. (2025, May 17). GeeksforGeeks. https://www.geeksforgeeks.org/dbms/introduction-of-er-model/

JUST A MOMENT... (n.d.). Just a moment... https://www.datacamp.com/tutorial/my-sql-tutorial

MYSQL CREATE DATABASE STATEMENT. (n.d.). W3Schools Online Web Tutorials. https://www.w3schools.com/mysql/mysql_create_db.asp

MYSQL TUTORIAL. (n.d.). W3Schools Online Web Tutorials. https://www.w3schools.com/MySQL/default.asp

(2023, December 30). MySQL Tutorial. https://www.mysqltutorial.org/

PAGE RESTRICTED. (n.d.). Page restricted | ScienceDirect. https://www.sciencedirect.com/science/article/abs/pii/B9780934613538500352